Go 语言能力考察题库 · 参考答案（对照版）

> 来源：个人思考 + 学习（配套 `go-internals-interview-questions-20260608.md` 的答案）
> 收录时间：2026-06-09 11:00 CST
>
> 对标：`CLAUDE.md` 能力地图 **格 1（语言与运行时）**。
> 用法：盖住本文先自己讲一遍，再逐题对照。答案力求「白板能讲 + 扛住三层追问」，但以理解为主，数字（扩容因子、对齐字节）记机制不必死背。
> 版本基准：以 **Go 1.22** 为主线，涉及版本差异处单独标注。

A. 语法与类型系统基础

1. 值传递还是引用传递?
Go **只有值传递**，没有引用传递。函数传参永远是拷贝一份实参的值。

- slice/map/channel 之所以「看起来像引用」，是因为它们本身是**含指针的结构体**（slice 是 `{ptr, len, cap}`，map/channel 是指向底层结构的指针）。拷贝这个 header 时底层数据指针被一起拷过去，所以函数内通过指针改底层数据，外部能看到。
- 但改 header 本身（如对 slice 重新赋值、append 导致换底层数组）外部看不到，因为拷贝的是 header。
- **追问（append）**：`func f(s []int){ s = append(s,1) }` 外部**看不到**新元素——因为 append 可能扩容换数组，且即使没扩容，新的 len 写在函数内的副本 header 上，外部 header 的 len 没变。分情况：① 未扩容时数据确实写进了共享底层数组的 `s[len]` 位置，但外部 len 没增长所以访问不到；② 扩容时连数据都写到新数组了。要让外部看到，得返回新 slice 或传 `*[]int`。

2. `var s []int` vs `s := []int{}`

- `var s []int`：**nil slice**，`s == nil` 为 true，ptr/len/cap 全 0。
- `s := []int{}`：**非 nil 空 slice**，指向一个长度为 0 的底层数组（runtime 的 `zerobase`），`s == nil` 为 false。
- 对 nil slice：`len`/`cap` 返回 0，`range` 不进循环，`append` 完全正常（会分配新数组）。**唯一区别是 `== nil` 和 JSON 序列化**（nil slice 序列化成 `null`，空 slice 成 `[]`）。习惯上优先用 `var s []int`。

3. 数组 vs slice

- 数组是**值类型、定长**，`[3]int` 和 `[4]int` 是不同类型；赋值/传参整体拷贝；长度是类型的一部分。
- slice 是**引用语义的 header**，变长，三个字段：`array unsafe.Pointer`（指向底层数组）、`len int`、`cap int`。slice 是对底层数组某段的「视图」。

4. `new` vs `make`

- `new(T)` 给任意类型分配零值内存，返回 `*T`。
- `make` 只用于 slice/map/channel，返回的是**初始化好的类型本身**（不是指针）。
- 为什么只能这三者：它们都是需要 runtime 初始化内部结构（slice 的底层数组、map 的 hmap、channel 的 hchan）才能用的「引用类型」，`new` 只清零内存不做初始化——`new(map[int]int)` 得到的是指向 nil map 的指针，不能直接写。

5. 零值机制
声明未显式初始化的变量自动赋零值：数值 0、bool false、string `""`、指针/slice/map/channel/func/interface 为 nil。struct 零值 = 每个字段递归取零值（不是 nil，是一个各字段都是零值的实例）。**nil 的类型**：pointer、slice、map、channel、func、interface。

6. iota
`iota` 是 const 块里的行计数器，每个 `const(...)` 块从 0 开始，每行 +1。

```
const (
    _  = iota             // 0，丢弃
    KB = 1 << (10 * iota) // 1<<10
    MB                    // 1<<20
    GB                    // 1<<30
)
```

7. 常量 vs 变量
常量在**编译期**确定，只能是基础类型（数值/字符串/bool）和无类型常量，不占运行时地址。**没有 const 指针/const 切片**——指针和切片是运行时构造，无法作为编译期常量。Go 没有 C++ 的 `const` 修饰符语义。

8. int 大小 / rune / byte
`int`/`uint` 是平台相关：32 位平台 4 字节，64 位平台 8 字节。`byte` 是 `uint8` 的别名，`rune` 是 `int32` 的别名（表示一个 Unicode 码点）。

9. 转换 vs 断言

- 类型转换 `T(x)`：编译期已知两类型可转（数值互转、`[]byte`↔`string` 等），针对**具体类型**。
- 类型断言 `i.(T)`：运行时从**接口值**里取出具体类型。`i.(T)` 单返回值断言失败 panic；`i.(T)` 双返回值 `v, ok := i.(T)` 失败返回零值+false；`i.(type)` 只能用在 `switch` 里做 type switch。

10. 字符串能否改字节
不能。`s[0]='x'` **编译报错**（string 索引不可寻址赋值）。string 底层是 `{ptr, len}` 指向**只读**字节数组，设计为不可变，这样可以安全共享底层内存（子串、`[]byte`→string 优化），也让 string 能做 map key、并发安全。要改先转 `[]byte`。

B. slice / map / string 底层

11. len vs cap
len 是当前元素个数，cap 是从 slice 起点到底层数组末尾的容量。`make([]int,3,10)`：len=3，cap=10，可索引 [0,3)，append 到第 8 个才扩容。

12. 扩容规则（讲机制）
append 超过 cap 时分配新数组并拷贝。经典规则：**旧 cap < 阈值时翻倍，超过阈值后约 1.25 倍**增长。Go 1.18 把阈值从 1024 改为 256，并在小到大之间做了平滑过渡（不再是硬跳变）。最终算出的容量还会按 size class 向上对齐到 runtime 的内存规格。要点：**小切片倍增**摊销拷贝成本，**大切片放缓**避免浪费内存。

13. 经典坑：`b := a[1:3]`
`a=[1,2,3,4,5]`，`b=a[1:3]` → b=`[2,3]`，len=2，cap=4（从下标1到数组末尾）。

- 对 b `append` 时若没超 cap（这里 cap=4，可再加 2 个），会**写进 a 的底层数组**，把 `a[3]`、`a[4]` 覆盖掉——改到了 a。
- 一旦 append 超过 cap=4，触发扩容换新数组，之后再改就**不影响** a。
- 防御写法：`b := a[1:3:3]`（三索引把 cap 限到 2），任何 append 都立即扩容，切断共享。

14. 安全删除中间元素

```
s = append(s[:i], s[i+1:]...) // 保序，O(n) 拷贝
s[i] = s[len(s)-1]; s = s[:len(s)-1] // 不保序，O(1)，但打乱顺序
```

陷阱：① 删除含指针元素时，被「挤掉」的尾部元素仍被底层数组引用，可能内存泄漏——需手动把尾部置零（`s[len(s)-1]=nil`）；② 循环里边删边遍历下标会错位。

15. 切断共享
`append([]T{}, s...)` 和 `copy` 都能拷贝出独立底层数组。`copy` 更稳/更明确：`dst := make([]T, len(s)); copy(dst, s)` 一次分配到位、长度精确；`append([]T{}, s...)` 从 0 cap 开始可能多次扩容。两者都只是浅拷贝（元素含指针时指向同一对象）。

16. map 底层 / 无序
map 底层是 `hmap`，含一组 `bucket`（`bmap`），每个 bucket 存 8 个 kv（开放定址 + 链式 overflow）。key 哈希后高 8 位选 bucket 内槽位、低位选 bucket。**遍历无序是设计**：runtime 故意每次从随机 bucket、随机槽位起始遍历，防止程序员依赖顺序（顺序本就因扩容 rehash 而变）。

17. map 并发
map **不支持并发读写**。并发写（或一写一读）runtime 检测到会直接 `fatal error: concurrent map read and map write` 崩溃（不可 recover）。方案：① `sync.RWMutex`+map（通用）；② `sync.Map`（读多写少、key 稳定）；③ 分片加锁（sharded map，降低锁竞争）；④ 用 channel 串行化访问。取舍见 G/Q 类。

18. 为什么 `m["k"].field = 1` 不行
map 的 value 不可寻址（map 可能 rehash 搬移 value，地址不稳定），不能取址改字段。绕法：① value 用指针 `map[string]*T`，`m["k"].field=1`；② 取出改完再塞回 `v:=m["k"]; v.field=1; m["k"]=v`。

19. map key 要求
key 必须**可比较**（支持 `==`）。可：基础类型、指针、channel、接口、元素可比较的数组、字段全可比较的 struct。不可：slice、map、func（不可比较）。所以 slice 不能做 key，元素全可比较的 struct 可以。

20. 删元素后内存释放?
`delete` 不缩容，map 的 bucket 数组**永不收缩**，删大量元素后内存仍占着（只是标记空槽）。要真正回收只能**重建**：新建一个 map 把要保留的拷过去，让老 map 被 GC。（注：value 含指针的话 delete 会清掉对 value 的引用，value 本身能被 GC，但 bucket 骨架不缩。）

21. string ↔ []byte 拷贝
标准转换**都会拷贝**（因为 string 不可变，必须复制以隔离可变的 []byte）。零拷贝可用 `unsafe`（见 Q137）或 1.20+ 的 `unsafe.String`/`unsafe.Slice`，代价：转出的 string 共享了 []byte 内存，若之后改了 []byte 会破坏 string 不可变假设，引发诡异 bug。

22. `range str` 的 c
`for i,c := range s` 中 c 是 **rune**（int32），range 按 UTF-8 解码。`i` 是**字节索引**（每个字符的起始字节位置），所以遍历中文时 i 会跳着走（0,3,6...）。若要按字节遍历用 `for i:=0;i<len(s);i++ { s[i] }`（s[i] 是 byte）。

C. 接口与方法集

23. iface vs eface

- `eface`（空接口 `any`）：`{_type *_type, data unsafe.Pointer}` 两个字。
- `iface`（带方法的接口）：`{tab *itab, data unsafe.Pointer}`。`itab` 里有接口类型、具体类型、方法地址表（用于动态派发）。
- data 指向堆/栈上的具体值（小值也可能装箱到堆，见 27）。

24. nil 接口陷阱

```
func f() error { var p *T = nil; return p } // f() != nil !
```

接口值非 nil 的条件是 **type 和 data 都为 nil**。把一个「类型为 `*T`、值为 nil」的指针赋给 error 接口后，接口的 type 字段=`*T`（非空），data=nil，所以 `err != nil` 成立。这就是返回 `*T` 给 error 的经典坑。**修复**：函数返回类型直接写 `error` 并显式 `return nil`，不要返回具体指针类型。

25. 方法集差异

- 值接收者 `func(t T)`：`T` 和 `*T` 都拥有该方法。
- 指针接收者 `func(t *T)`：**只有** `*T` 拥有该方法，`T` 的方法集不含它。
- 所以：若接口方法用指针接收者实现，只有 `*T` 满足接口，`T` 不满足。

26. 值变量能否调指针方法
能，但要求**可寻址**。`var t T; t.PtrMethod()` 编译器自动取址 `(&t).PtrMethod()`。不可寻址的值不行：map 的 value（`m["k"].PtrMethod()` 报错）、函数返回值、常量。

27. any 的代价

- **装箱**：值塞进 eface 要带类型信息，值本身常被分配到堆（逃逸）。
- **拆箱**要类型断言/反射，有运行时开销，且把类型检查从编译期推到运行期，丢失类型安全（可能 panic）。
- 1.18 后能用泛型表达的就别用 any。

28. 动态派发开销
接口方法调用通过 itab 的函数指针间接调用，比直接调用多一次指针解引用，且**通常无法内联**，妨碍编译器优化。开销在热路径上可观测，但绝大多数场景可忽略。编译器在能静态确定具体类型时（去虚拟化 devirtualization）有时能内联。

29. 判断底层类型 / type switch

```
switch v := i.(type) {
case int: ...      // v 是 int
case *T: ...
default: ...
}
```

30. 泛型 vs 接口

- 换成泛型：**容器/算法**（`Map`、`Filter`、`Min`、数据结构），原来用 `interface{}` 装载丢类型的场景——泛型保留具体类型、避免装箱、零运行时断言。
- 仍用接口：需要**运行时多态/异质集合**（一个 slice 装不同实现）、解耦依赖（如 `io.Reader`）。
- 取舍：泛型是「编译期同一行为不同类型」，接口是「运行时不同实现统一抽象」。

31. 嵌入 vs 继承
嵌入是**组合不是继承**，没有父子/多态关系。

- 嵌入 struct：外层自动获得内层的字段和方法（**方法提升**），可直接 `outer.InnerMethod()`；但 `outer` 不是 `inner` 的子类型，不能赋值给 inner 类型。
- 嵌入接口：外层接口拥有被嵌入接口的所有方法（如 `io.ReadWriter` 嵌入 `Reader`+`Writer`）。
- 提升规则：浅层优先，冲突时需显式指定路径；提升的方法的接收者仍是内层类型。

D. defer / panic / recover / error

32. defer 执行顺序
后进先出（栈）。同一函数多个 defer，逆序执行。

33. 口算：命名返回值 + defer

```
func f() (r int) { defer func(){ r++ }(); return 5 } // 返回 6
```

`return 5` 先把 5 赋给命名返回值 r，再执行 defer，defer 里 `r++` 改的就是这个 r，最终返回 6。
换非命名返回值 `func f() int { ... return 5 }`：defer 改不到返回值（返回值是匿名临时，已拷贝），返回 5。**机制**：`return x` = 「赋值给返回变量 → 执行 defer → 真正返回」三步。

34. defer 参数求值时机
defer 语句执行时立即求值参数，只是函数调用延迟。`for i:=0;i<3;i++ { defer fmt.Println(i) }` 打印 `2 1 0`（i 在 defer 时就被拷贝）。若想延迟求值用闭包 `defer func(){ fmt.Println(i) }()`。

35. defer 开销 / 优化
早期 defer 走堆分配的 `_defer` 链表，较慢。Go 1.13 引入栈分配 defer，1.14 引入**开放编码 defer（open-coded defer）**：编译期把 defer 直接内联到函数尾部，开销接近普通函数调用，几乎可忽略。但**循环里的 defer**或数量不定时无法开放编码，退回较慢路径。

36. recover 生效位置
只在 defer 直接调用的函数里生效。`defer recover()` 直接写无效（recover 不能在 defer 调的函数再下一层），必须 `defer func(){ recover() }()`。原因：recover 要捕获的是当前 goroutine 正在 panicking 的栈帧。

37. 跨 goroutine recover
不能。panic 只在当前 goroutine 的栈上展开，recover 必须在同一 goroutine 的 defer 里。子 goroutine panic 没被自己 recover 会导致**整个进程崩溃**。工程意义：每个可能 panic 的 goroutine 都要自带 recover（如 worker 包一层），不能指望主 goroutine 兜底。

38. error wrapping
1.13 的 `%w` 让 error 可以**包装并保留底层错误链**：`fmt.Errorf("...: %w", err)`。

- `errors.Is(err, target)`：沿链判断是否**等于/匹配**某个哨兵错误。
- `errors.As(err, &target)`：沿链找出第一个能**赋值给某类型**的错误，取出来。
- 解决了过去 `fmt.Errorf("%v")` 丢失原始错误类型、无法判断根因的问题。

39. `errors.Is` vs `==`
`==` 只比较当前这一层；`errors.Is` 会**解包整条 wrap 链**逐层比，还会调用错误的 `Is(error) bool` 方法做自定义匹配。自定义 error 支持 Is：实现 `func (e *MyErr) Is(target error) bool`。

40. panic vs error 边界

- error：**可预期、可恢复**的错误（文件不存在、网络超时、参数非法），是控制流的一部分，必须返回让调用方处理。
- panic：**不可恢复的程序 bug / 违反不变量**（数组越界、空指针、不该到达的分支），表示「程序状态已损坏」。
- 库代码原则上不 panic（除非真正不可恢复），通过 error 表达；panic 留给 main 或顶层中间件兜底。

41. HTTP panic 恢复中间件

```
func Recover(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        defer func() {
            if e := recover(); e != nil {
                log.Printf("panic: %v\n%s", e, debug.Stack())
                http.Error(w, "internal error", 500)
            }
        }()
        next.ServeHTTP(w, r)
    })
}
```

注意：每个请求的 handler 在自己的 goroutine 里跑，所以这个 defer 能 recover 该请求 goroutine 的 panic；但 handler 内部又开的子 goroutine panic 它救不了。

E. goroutine 与并发模型

42. goroutine vs 线程
goroutine 是用户态轻量协程，由 Go runtime 调度（M:N 映射到 OS 线程），创建/切换成本远低于线程（无内核态切换）。初始栈 **2KB**（线程通常 1-8MB）。栈按需增长：**连续栈 + 栈拷贝**——栈不够时分配一块更大的（翻倍）、把旧栈整体拷过去、改指针（早期是分段栈，会有「热分裂」问题已弃用）。

43. 能开多少 / 成本
理论上百万级。主要成本是**内存**（每个起步 2KB，百万≈2GB+）和**调度/GC 扫描栈**的开销。瓶颈通常不是数量而是它们抢的资源（锁、IO、CPU）。

44. 经典坑：循环变量捕获
Go 1.22 **之前**，`for _,v:=range list` 的 v 是**整个循环复用的同一变量**，`go func(){ use(v) }()` 闭包捕获的是 v 的地址，goroutine 真正跑时 v 可能已被改成最后一个值——典型「全部打印最后一个元素」。修复：① `v:=v` 重新声明；② 当参数传 `go func(v T){...}(v)`。**Go 1.22 起**循环变量每轮迭代是新变量，这个坑基本消除（但显式传参仍是好习惯）。

45. goroutine 泄漏
goroutine 永久阻塞、无法退出即泄漏。三种典型：① 向无人接收的 channel 发送/从无人发送的 channel 接收且无 close；② `for{ select{} }` 没有退出分支、没监听 ctx.Done()；③ WaitGroup 计数错误导致永久 Wait。检测：`pprof` 的 goroutine profile（`/debug/pprof/goroutine`）看数量随时间增长，dump 看卡在哪一行。

46. 主 goroutine 退出 / 等待
`main` 返回**整个进程立即退出**，其它 goroutine 被强杀（不执行 defer）。等所有结束用 `sync.WaitGroup`（Add/Done/Wait）或 errgroup。

47. 限并发 / channel 信号量

```
sem := make(chan struct{}, N) // 容量 N 的信号量
for _, task := range tasks {
    sem <- struct{}{}         // 获取
    go func(t Task){ defer func(){ <-sem }(); do(t) }(task)
}
```

48. WaitGroup 误用

- `Add` 必须在对应 goroutine **启动之前**调用，放在 goroutine 内部是 bug：Wait 可能在 Add 执行前就返回（计数还是 0），漏等。
- Done 多调（负计数）panic；少调则 Wait 永久阻塞。
- 一般 `wg.Add(1)` 在 `go` 前，goroutine 内 `defer wg.Done()`。

49. errgroup 的价值
WaitGroup 不能传错误、不能取消。`errgroup.Group` 在等待一组 goroutine 的基础上：① 收集**第一个非 nil error**；② `WithContext` 在任一失败时**自动 cancel** 其它任务；③ `SetLimit` 限并发。

50. 并发 vs 并行 / GOMAXPROCS
并发是「同时处理多件事的能力」（结构），并行是「同一时刻真在跑多件」（执行）。`GOMAXPROCS` 控制能同时执行 Go 代码的 **P（逻辑处理器）数量**，即并行度。设成 1 仍能并发（goroutine 在单个 P 上轮流跑），只是不并行。

F. channel

51. hchan 结构 / 有无缓冲
`hchan` 含：环形缓冲区 `buf`、容量 `dataqsiz`、当前元素数 `qcount`、发送/接收等待队列 `sendq`/`recvq`（goroutine 链表）、`lock`。无缓冲 channel `dataqsiz=0`，发送方必须等接收方就绪（同步握手）；有缓冲在 buf 未满时发送不阻塞。

52. nil / 已关闭 channel 操作

- nil channel：读、写都**永久阻塞**（常用于 select 中动态禁用某分支）。
- 已关闭 channel：**读**立即返回（先取完剩余缓冲值，再返回零值+`ok=false`）；**写 panic**；**重复 close panic**。

53. 判断关闭
`v, ok := <-ch`；`ok==false` 表示 channel 已关闭且缓冲已空。range channel 会在 close 后自动结束循环。

54. 谁负责 close
发送方 close，且由唯一发送方 close。原则「接收方不 close、不重复 close」因为：close 是「不再有值发送」的信号，接收方无从判断；多发送方时谁都不该单独 close（会让其他发送方写已关闭 channel panic）。重复 close panic。多发送方场景用额外的 done channel 或 sync.Once 协调。

55. 无缓冲发送/接收阻塞 & happens-before
无缓冲 channel 发送和接收是**同步会合**：先到的一方阻塞等另一方。语义保证：**发送 happens-before 对应接收完成**（实际上接收完成 happens-before 发送返回）。常用作同步信号。

56. select 随机性 / default / 超时
多个 case 同时就绪时 select **伪随机**选一个（防饥饿）。`default` 让 select 非阻塞（无就绪 case 立即走 default）。超时：

```
select {
case v := <-ch: use(v)
case <-time.After(time.Second): // 超时
}
```

（生产中 `time.After` 在循环里会泄漏 timer，宜用 `time.NewTimer` + Stop。）

57. 骨架

```
// 生产者-消费者
ch := make(chan int, 100)
go func(){ for _,x:=range data { ch<-x }; close(ch) }() // 生产者 close
for v := range ch { consume(v) }                          // 消费者 range

// fan-out：多 worker 读同一 ch；fan-in：多 ch 合并到一个 out
func fanIn(chs ...<-chan int) <-chan int {
    out := make(chan int); var wg sync.WaitGroup
    for _, c := range chs { wg.Add(1); go func(c <-chan int){ defer wg.Done(); for v:=range c { out<-v
} }(c) }
    go func(){ wg.Wait(); close(out) }()
    return out
}

// 优雅关闭一组 worker：广播 done
done := make(chan struct{})
for i:=0;i<N;i++ { go func(){ for { select{ case t:=<-tasks: do(t); case <-done: return } } }() }
close(done) // 广播退出
```

58. channel vs mutex

- channel 适合**传递数据所有权 / 协调 goroutine / 编排流水线**（通信）。
- mutex 适合**保护一小段共享状态**（计数器、缓存、map），简单且更快。
- 「Don't communicate by sharing memory」是倡导优先用 channel 表达协作；但当只是保护一个共享变量、没有所有权转移时，**mutex 更直接更高效**，强行 channel 反而绕。

59. 并发安全可超时对象池

```
type Pool struct{ ch chan *Obj }
func New(n int) *Pool { p:=&Pool{ch:make(chan *Obj,n)}; for i:=0;i<n;i++{p.ch<-newObj()}; return p }
func (p *Pool) Get(ctx context.Context) (*Obj,error) {
    select { case o:=<-p.ch: return o,nil; case <-ctx.Done(): return nil,ctx.Err() }
}
func (p *Pool) Put(o *Obj){ p.ch<-o }
```

60. close 广播 vs 发 N 个值
close 一次性唤醒**所有**在该 channel 阻塞的接收者（且后续接收都立即返回），不需要知道接收者数量；发 N 个值要精确知道 N，且无法重复通知。context 的 cancel 底层正是 close 一个 done channel 实现广播。

G. sync 包与内存模型

61. Mutex vs RWMutex
RWMutex 允许多读并发、写独占。读多写少时 RWMutex 才更快；读写均衡或临界区极短时，RWMutex 的额外记账开销可能**比 Mutex 还慢**。坑：**写饥饿**——持续不断的读会让写者长期拿不到锁（Go 的 RWMutex 有缓解：一旦有写者等待，后续读者排在其后）。

62. Mutex 公平性 / 两种模式
Go Mutex 不是严格公平锁，有两种模式：

- **正常模式**：等待者排 FIFO 队列，但新来的 goroutine 正在 CPU 上有优势，可能直接抢到锁（吞吐高，但可能让队首饥饿）。
- **饥饿模式**：当某等待者超过 1ms 没拿到锁，切到饥饿模式，锁直接移交队首，新来者不抢、直接排队尾（保公平，牺牲吞吐）。饥饿缓解后切回正常模式。

63. sync.Once
保证函数只执行一次。底层：一个 `done uint32` 标志 + Mutex。**双检**：先原子读 done 快速路径（已完成直接返回），未完成才加锁、再检查一次 done、执行、原子置 done。1.21 改用 atomic.Uint32。保证：Once.Do 返回前函数已执行完（happens-before），并发调用其他 goroutine 会阻塞等第一个完成。

64. WaitGroup 复用
能复用，但**必须在上一轮 Wait 返回后**才能开始下一轮 Add。Wait 还没返回时 Add（计数从 0 涨起）有 data race 风险、行为未定义。常见模式是每批用完重新 Add。

65. sync.Pool
缓存临时对象、减少 GC 压力和分配（如 buffer 复用）。对象回收时机：**每次 GC 会清空 Pool**（准确说 1.13+ 用 victim cache，对象最多存活两个 GC 周期）。注意：① 不能假设 Put 进去的东西 Get 还能拿到（随时被回收）；② Get 出来的对象状态不确定，要 **Reset**；③ 适合无状态、可重置的临时对象，不适合连接等长生命周期资源。

66. atomic / atomic.Value / CAS / ABA
`atomic` 提供无锁原子操作（Add/Load/Store/Swap/CompareAndSwap）。`atomic.Value` 适合**整体替换**的配置/快照（读多写少，无锁读）。CAS：比较并交换，是乐观锁基础。ABA：值从 A→B→A，CAS 误以为没变；Go 里大多数场景（如计数、指针替换）不受 ABA 影响，需要担心时用版本号/带标记指针——但 Go 实践中很少手写到那个层面。

67. 内存模型 / happens-before
没有同步时，编译器和 CPU 可**重排指令**、各核有自己的缓存，写未必及时对其他核可见，于是读到旧值；非原子的大值（如 interface、slice header）读写非原子，可能读到「写了一半」的撕裂值。**happens-before** 是 Go 内存模型定义的偏序：若操作 A happens-before B，则 A 的效果对 B 可见。同步原语（channel、Mutex、Once、atomic）建立 happens-before 边。**结论：共享变量必须用同步原语，否则行为未定义。**

68. data race vs race condition
不是一回事。**data race**：两个 goroutine 并发访问同一内存、至少一个是写、且无同步——是内存层面的未定义行为。**race condition**：程序逻辑因执行时序不同产生错误结果（可能没有 data race，比如先查后改的 TOCTOU）。`-race` 检测器只查 **data race**，查不出逻辑层 race condition。

69. DCL 裸 if
不能用裸 if 实现双重检查锁。`if instance==nil` 的读和初始化写之间没有同步，有 data race，且可能读到「指针已设但对象未初始化完」的半成品（重排）。必须用 `sync.Once`（推荐）或 `atomic` 配合 acquire/release 语义。

70. 并发计数器 mutex vs atomic

```
// mutex
var mu sync.Mutex; var n int
func inc(){ mu.Lock(); n++; mu.Unlock() }
// atomic
var n int64
func inc(){ atomic.AddInt64(&n,1) }
```

性能差异来源：atomic 是单条 CPU 原子指令（LOCK 前缀），无需进出临界区、无 goroutine 阻塞/唤醒；mutex 在竞争时涉及排队、可能陷入内核休眠。单变量自增 atomic 明显更快；保护多个变量/复杂逻辑只能用 mutex。

71. 伪共享 / padding
伪共享：两个本无关的变量落在同一 **cache line（通常 64 字节）**，不同核分别频繁写，导致 cache line 在核间反复失效/同步，性能骤降。缓解：用 padding 把高频写的变量隔到不同 cache line：

```
type Counter struct { n int64; _ [56]byte } // 凑满 64 字节
```

H. context

72. context 解决什么 / 四个派生函数
解决**请求级的取消、超时、截止时间传播和请求作用域值传递**，让一个请求链路上的所有 goroutine 能被统一取消、避免泄漏。

- `WithCancel`：返回可手动取消的 ctx + cancel 函数。
- `WithTimeout`：相对超时（如 5s 后）自动取消。
- `WithDeadline`：绝对时间点截止。
- `WithValue`：挂载请求作用域的键值（如 traceID）。

73. 取消传播机制
底层是 **channel + 树形父子结构**。每个可取消 ctx 有个 `done chan struct{}`，取消时 `close(done)` 广播。父 ctx 取消会**递归取消所有子 ctx**（父维护 children 集合，cancel 时遍历调子的 cancel）。监听方通过 `<-ctx.Done()` 感知。所以是 channel（done）+ 回调式的父子联动两者结合。

74. WithValue 用法
该传：**请求作用域的元数据**（traceID、用户身份、认证 token）——横切关注点。不该传：**函数的必要业务参数**（应显式作形参，否则隐藏依赖、类型不安全、难测）。key 用**自定义私有类型**（`type ctxKey int`）而非 string，避免不同包的 key 字符串碰撞、防止外部包覆盖你的 key。

75. 不调 cancel 的后果
`WithCancel/WithTimeout/WithDeadline` 会向父 ctx 注册 child，并（超时类）启动 timer。不调 cancel 则 child 一直挂在父的 children 里**无法被 GC**，timer 也可能不释放——goroutine/内存泄漏。所以 `ctx, cancel := ...; defer cancel()` 几乎是强制写法（即使超时已触发，cancel 也幂等、用于清理）。

76. 取消后会强杀 goroutine 吗
不会。context 取消只是**关闭 done channel 发信号**，不能抢占/杀死 goroutine。必须由 goroutine 自己**主动监听** `ctx.Done()` 并返回（或把 ctx 传给支持取消的阻塞调用如 DB/HTTP）。不监听就取消不掉——这是协作式取消。

77. ctx 贯穿链路
HTTP 入口 `r.Context()` 拿到请求 ctx（客户端断开会自动取消）→ 传给 service 层 → 传给 DB 驱动（`db.QueryContext(ctx, ...)`）。每层可 `WithTimeout` 再派生更短超时。超时逐层生效：任一层 ctx 到期，其 done 关闭，正在该 ctx 上等待的调用（DB 查询、下游 RPC）感知后提前返回 `context.DeadlineExceeded`。

78. ctx.Err() 两种返回值

- `context.Canceled`：被主动 cancel。
- `context.DeadlineExceeded`：超时/到期。

未取消时返回 nil。

I. 调度器（GMP）

79. GMP 模型

- **G（goroutine）**：一个协程，含栈、PC、状态。
- **M（machine）**：OS 线程，真正执行代码的实体。
- **P（processor）**：逻辑处理器，是 G 和 M 之间的「调度上下文」，持有可运行 G 的本地队列、mcache 等资源。数量 = GOMAXPROCS。
- M 必须**绑定一个 P** 才能执行 G。为什么需要 P：早期 G-M 模型用全局队列+全局锁，竞争严重。引入 P 后每个 P 有本地运行队列，调度本地化、几乎无锁，且 P 数量限制并行度、隔离 mcache，极大提升可扩展性。

80. 本地队列 vs 全局队列 / work-stealing
每个 P 有容量 256 的本地队列（runnext + 环形队列）；放不下或定期会丢到全局队列。当一个 P 的本地队列空了，它会：① 查全局队列；② 查 netpoller；③ **从其他 P 偷一半 G**（work-stealing），保证负载均衡、CPU 不空转。

81. 抢占时机 / 协作式 → 异步抢占
切换时机：channel 阻塞、系统调用、`go`/GC、函数调用时的栈检查点、以及被抢占。Go 1.14 前是**协作式抢占**：只在函数调用的栈增长检查点插入抢占检查，所以**没有函数调用的纯计算死循环**永远不让出，会卡住 GC 和调度（甚至整个程序）。1.14 起改为**基于信号的异步抢占**：sysmon 监控线程发现某 G 跑太久（>10ms），向其 M 发 `SIGURG` 信号，在信号处理里安全点抢占，解决了死循环卡死问题。

82. 阻塞系统调用 / hand off
G 进入**阻塞式系统调用**（如读文件）时，M 会被系统调用阻塞，runtime 把该 M 持有的 **P 解绑（hand off）**给其他空闲 M（或新建 M），让 P 上的其他 G 继续在别的线程跑。系统调用返回后，原 M 试图拿回一个 P，拿不到就把 G 放回队列、自己休眠。**网络 IO 不同**：走 netpoller，不会真的阻塞 M（见下）。

83. netpoller
netpoller 是 runtime 基于 OS 多路复用（Linux epoll / macOS kqueue / Windows IOCP）的网络轮询器。你写「阻塞式」网络读 `conn.Read`，runtime 实际把 fd 设为非阻塞、注册到 epoll，然后**把当前 G 挂起（park）**让出 M，M 去跑别的 G；当 fd 就绪，netpoller 唤醒对应 G 重新入队。于是看似阻塞的代码不占用 OS 线程，能支撑海量并发连接。

84. GOMAXPROCS 默认 / 容器
默认 = `runtime.NumCPU()`（机器逻辑核数）。**在容器里**（用 cgroup 限了 CPU，如 limit 2 核但宿主 64 核），NumCPU 看到的是宿主 64，GOMAXPROCS 默认错误地设成 64，导致过多 P、过度上下文切换和调度开销。`uber-go/automaxprocs` 读 cgroup 的 CPU quota 把 GOMAXPROCS 设成实际配额。（注：Go 1.25 已让 runtime 默认感知 cgroup 配额。）

85. runtime.Gosched()
主动让出当前 P，把 G 放回队列尾，让其他 G 有机会跑（不阻塞，会被重新调度）。现代 Go 有异步抢占后很少需要手动调用，极少数极端紧密循环里可能有意义，一般不用。

86. 无函数调用的死循环
老版本（<1.14）会卡死整个程序：协作式抢占只在函数调用检查点抢占，纯死循环没有检查点，永远不让出 P，若 GOMAXPROCS=1 还会阻止 GC（GC 要等所有 G 到安全点）导致整个程序挂起。1.14 异步抢占后不再卡死。

J. 内存管理与 GC

87. 分配器分层
仿 TCMalloc 的多级缓存：

- **mcache**：每个 P 一个，**无锁**，分配小对象的快路径。
- **mcentral**：每个 size class 一个，全局，mcache 不够时来这里要 span（带锁）。

- **mheap**：管理整个堆，向 OS 申请内存（以 page 为单位），mcentral 不够找它。
- 对象按大小分：**tiny（<16B 无指针）**合并分配、**small（≤32KB）**按 size class 从 mcache 取、**large（>32KB）**直接从 mheap 分配。
- 分级的意义：小对象走 per-P 无锁缓存，避免全局锁竞争；按 size class 减少碎片。

88. GC 算法 / 三色不变性
并发三色标记 + 混合写屏障的标记-清除（非分代、非移动）。三色：白（未扫描，待回收）、灰（已发现待扫描其引用）、黑（已扫描完）。从根（栈、全局）出发把可达对象逐步由白→灰→黑。**强三色不变性**：黑对象不能直接指向白对象。标记结束仍是白的对象即垃圾。

89. 写屏障
并发标记时**用户程序还在改指针**，可能出现「黑对象新指向一个白对象，且原引用被删」，导致白对象被漏标误删。写屏障是在指针写操作时插入的一小段代码，把涉及的对象**着灰/标记**，维持三色不变性。Go 1.8 起用**混合写屏障**（Yuasa 删除屏障 + Dijkstra 插入屏障结合），好处是**栈不需要二次 STW 重扫**。

90. STW 阶段
现代 Go GC 有两次极短 STW：**Mark Setup**（开启写屏障、准备标记，~微秒级）和 **Mark Termination**（关闭写屏障、收尾）。主体的**并发标记**和**并发清扫**与用户程序同时进行。STW 通常 <1ms（亚毫秒），目标是低延迟。

91. GOGC / GOMEMLIMIT

- `GOGC`（默认 100）：控制触发下次 GC 的堆增长比例。100 表示堆相比上次 GC 后存活量增长 100%（翻倍）时触发。调大 → GC 频率低、内存占用高、吞吐高；调小 → GC 勤、省内存、CPU 开销大；设 off 关闭 GC。
- `GOMEMLIMIT`（1.19+）：软性内存上限。即使按 GOGC 还没到触发点，接近该上限时也会更激进 GC，**防 OOM**。适合容器里设成略低于内存 limit，避免被 OOMKill。两者配合：GOGC 控常态节奏，GOMEMLIMIT 兜底。

92. 内存泄漏判断
Go 有 GC，「泄漏」通常指**对象一直被可达引用而无法回收**：goroutine 泄漏（其栈和引用的对象都不回收）、全局 map 只增不删、未关闭的 timer/ticker、slice 截取持有大底层数组、未调 cancel 的 ctx。定位：`pprof` heap profile 看谁占内存、`inuse_space` 增长趋势；goroutine profile 看 goroutine 数是否持续涨。

93. GC 触发时机 / 手动 GC
触发：① 堆增长达 GOGC 阈值；② 距上次 GC 超过 2 分钟的强制触发；③ 接近 GOMEMLIMIT；④ `runtime.GC()` 手动。`runtime.GC()` 几乎不该在生产逻辑里用（会 STW、打乱节奏），仅用于测试、benchmark 前清理、或导出 heap profile 前确保准确。

94. 内存对齐 / struct 大小口算
CPU 按对齐边界访问内存更高效，编译器按字段类型的对齐要求**填充（padding）**。字段顺序影响 padding：把大字段排前、小字段聚拢能省内存。例（64 位）：

```
struct{ a bool; b int64; c bool } // 1 +(7 pad)+ 8 + 1 +(7 pad) = 24 字节
struct{ b int64; a bool; c bool } // 8 + 1 + 1 +(6 pad) = 16 字节
```

口算：按字段顺序累加，每个字段起始偏移要对齐到自身大小，末尾按最大对齐补齐。

K. 逃逸分析与性能

95. 逃逸分析 / 栈 vs 堆
逃逸分析是编译器静态判断变量的生命周期是否「逃出」当前函数作用域：不逃逸分配在**栈**（函数返回自动回收，无 GC 负担、分配=移动栈指针，极快），逃逸则分配在**堆**（需 GC 管理，分配慢）。性能差异来源：栈分配几乎零成本，堆分配要走分配器 + 增加 GC 扫描/回收压力。

96. 逃逸的典型情况
① 返回局部变量的指针；② 变量被赋给 interface（装箱）并发生动态调用；③ 闭包捕获并在外部存活的变量；④ 变量太大超过栈容量、或编译期大小不确定（如 `make([]T, n)` n 是变量）；⑤ 被发送到 channel 的指针/含指针对象；⑥ 调用 `fmt.Println(x)` 这类 `...interface{}` 参数常导致逃逸。

97. 返回局部变量指针
安全，和 C 截然不同。C 里返回栈上局部变量地址是悬垂指针（栈帧销毁）。Go 的逃逸分析检测到地址被返回，会自动把该变量**分配到堆**，由 GC 管理生命周期，所以安全。代价是这次分配逃到了堆。

98. 查看逃逸 / 读输出
`go build -gcflags='-m'`（`-m -m` 更详细）。输出如 `moved to heap: x`、`x escapes to heap`、`... does not escape`。读法：关注 "escapes to heap" / "moved to heap" 定位逃逸点，结合调用链判断原因。

99. []byte ↔ string 频繁转换优化
每次标准转换都拷贝。优化：① 尽量在 `[]byte` 或 `string` 一侧完成全部操作，减少来回转；② 用 `strings.Builder`（内部 []byte，最后零拷贝转 string）；③ map 查找时 `m[string(b)]` 编译器有特殊优化**不拷贝**；④ 极致场景用 `unsafe` / `unsafe.String` 零拷贝（需保证只读，见 Q137）。

100. 预分配
`make([]T, 0, n)` 一次分配到位，避免 append 过程中多次扩容+拷贝（每次扩容 O(n) 拷贝，累计开销大、产生垃圾）。map 预设容量 `make(map[K]V, n)` 减少 rehash 搬迁。不预分配的代价：多次重分配、内存拷贝、GC 压力、内存峰值翻倍。已知大小就该预分配。

101. pprof 五类 profile

- **cpu**：函数占用 CPU 时间，查热点/计算瓶颈。
- **heap**：内存分配/占用，查内存大户、泄漏。
- **goroutine**：当前所有 goroutine 栈，查泄漏/死锁/卡点。
- **block**：阻塞在同步原语（channel/mutex）的等待时间，查阻塞瓶颈。
- **mutex**：锁竞争，查热点锁。

102. benchmark 定位分配

```
func BenchmarkX(b *testing.B) {
    setup()
    b.ResetTimer()      // 排除 setup 的计时
    b.ReportAllocs()    // 报告每次操作的分配次数/字节
    for i:=0;i<b.N;i++ { X() }
}
```

`go test -bench=. -benchmem`。看 `allocs/op` 和 `B/op` 定位分配热点。`ResetTimer` 清掉准备阶段干扰，`ReportAllocs` 输出分配指标。

103. 字符串拼接性能排序
`strings.Builder` ≈ `bytes.Buffer` > `+`（少量、编译期常量拼接可优化）» `fmt.Sprintf`。原因：`+` 在循环里每次生成新 string 拷贝（$O(n^2)$）；`fmt.Sprintf` 还有反射/格式解析开销最慢；`strings.Builder` 内部用可增长 []byte、`String()` 零拷贝返回，最适合循环拼接（配合 `Grow` 预分配更佳）。`bytes.Buffer` 类似但 `String()` 会拷贝。

L. 工程化与工具链

104. go.mod / go.sum

- `go.mod`：声明 module 路径、Go 版本、直接/间接依赖及其版本。
- `go.sum`：记录每个依赖模块内容的**加密哈希校验和**。

- `go.sum` 防的是**供应链篡改**：下载依赖时校验哈希，确保拿到的代码和首次记录的一致，防止依赖被中间人篡改或仓库被悄悄改动（中间人攻击 / 依赖投毒）。

105. 语义化版本 / MVS / replace / exclude

SemVer：`v主.次.修`，主版本变更代表不兼容（Go module 用 `/v2` 后缀区分）。**MVS（最小版本选择）**：构建时对每个依赖选「所有要求中的最高的那个最低版本」，即满足所有约束的最小可行版本集——可重现、不会自动跳到未验证的新版（与 npm 的「装最新兼容版」相反）。`replace`：把某依赖替换成本地路径或 fork（本地开发、临时打补丁）；`exclude`：排除某个有问题的版本。

106. go vet / golangci-lint / -race / -cover

- `go vet`：静态检查可疑代码（Printf 格式串不匹配、锁拷贝、unreachable 等），不是风格而是潜在 bug。
- `golangci-lint`：聚合几十个 linter 的元工具，查风格、复杂度、潜在错误、性能问题。
- `-race`：运行时 data race 检测器。
- `-cover`：测试覆盖率统计。

107. build tag / 跨平台

文件顶部 `//go:build linux && amd64`（旧式 `// +build`）控制条件编译。文件名后缀也生效：`foo_linux.go`、`foo_windows.go`、`foo_amd64.go` 自动按平台编译。跨平台构建：`GOOS=linux GOARCH=arm64 go build`，Go 交叉编译开箱即用（纯 Go 无需目标平台工具链；cgo 除外）。

108. init 执行顺序

单包内：先初始化包级变量（按依赖顺序），再按**文件名字典序**依次执行各文件的 `init()`，同一文件内多个 init 按出现顺序。跨包：被依赖的包**先**初始化（深度优先，导入的包先于导入者），且每个包只初始化一次。`main` 包最后，所有依赖 init 完才执行 `main()`。

109. 依赖注入 / 为什么不流行重型 DI

DI 在 Go 里通常就是**构造函数显式传依赖**（接口参数），手动 wire。原因：Go 推崇显式、简单，接口是隐式实现、组合方便，编译期类型检查已能保证装配正确，不需要运行时反射式容器。需要管理复杂依赖图时用 Google 的 `wire`（编译期代码生成，无运行时开销）而非反射式框架，符合「无魔法」哲学。

110. mock 外部依赖 / table-driven

依赖定义成**接口**，生产用真实实现，测试传桩（stub）或 mock：

```
type Store interface{ Get(id int)(*User,error) }
type fakeStore struct{}
func (fakeStore) Get(id int)(*User,error){ return &User{ID:id}, nil }

func TestSvc(t *testing.T){
    tests := []struct{ name string; in int; want string }{
        {"ok", 1, "alice"}, {"empty", 0, ""},
    }
    for _, tt := range tests {
        t.Run(tt.name, func(t *testing.T){
            got := NewSvc(fakeStore{}).Name(tt.in)
            if got != tt.want { t.Errorf("got %q want %q", got, tt.want) }
        })
    }
}
```

可用 `gomock` / `mockery` 生成 mock。table-driven：用切片列 case，`t.Run` 子测试隔离、定位清晰。

111. graceful shutdown

```
srv := &http.Server{Addr: ":8080", Handler: mux}
go func(){ if err := srv.ListenAndServe(); err != http.ErrServerClosed { log.Fatal(err) } }()

ctx, stop := signal.NotifyContext(context.Background(), syscall.SIGINT, syscall.SIGTERM)
defer stop()
<-ctx.Done() // 等信号

shutCtx, cancel := context.WithTimeout(context.Background(), 10*time.Second)
defer cancel()
srv.Shutdown(shutCtx) // 停止接新连接，等存量请求处理完（或超时）
```

要点：`Shutdown` 不杀正在处理的请求，给 ctx 设超时上限兜底；信号触发后串起 signal → context → Shutdown。

M. 综合 / 手写题（白板）

112. 并发安全 LRU

```
type entry struct{ k, v int }
type LRU struct {
    mu   sync.Mutex
    cap  int
    ll   *list.List               // 双向链表，front=最近用
    m    map[int]*list.Element    // key -> 链表节点
}
func NewLRU(cap int) *LRU { return &LRU{cap:cap, ll:list.New(), m:map[int]*list.Element{}} }
func (c *LRU) Get(k int) (int, bool) {
    c.mu.Lock(); defer c.mu.Unlock()
    if e, ok := c.m[k]; ok { c.ll.MoveToFront(e); return e.Value.(*entry).v, true }
    return 0, false
}
func (c *LRU) Put(k, v int) {
    c.mu.Lock(); defer c.mu.Unlock()
    if e, ok := c.m[k]; ok { e.Value.(*entry).v = v; c.ll.MoveToFront(e); return }
    e := c.ll.PushFront(&entry{k, v}); c.m[k] = e
    if c.ll.Len() > c.cap {
        old := c.ll.Back(); c.ll.Remove(old); delete(c.m, old.Value.(*entry).k)
    }
}
```

要点：map O(1) 定位 + 双向链表 O(1) 维护使用顺序；Get/Put 都要把节点移到 front；超容删 back。

113. 令牌桶限流器

```
type TokenBucket struct {
    mu       sync.Mutex
    tokens   float64
    cap      float64
    rate     float64   // 每秒补充
    last     time.Time
}
func (b *TokenBucket) Allow() bool {
    b.mu.Lock(); defer b.mu.Unlock()
    now := time.Now()
    b.tokens += now.Sub(b.last).Seconds() * b.rate // 按时间差补充
    if b.tokens > b.cap { b.tokens = b.cap }
    b.last = now
```

```
    if b.tokens >= 1 { b.tokens--; return true }
    return false
}
```

令牌桶允许突发（攒满桶可瞬时放 cap 个）；漏桶按固定速率流出、平滑无突发。生产直接用 `golang.org/x/time/rate`。

114. 两 goroutine 交替打印

```
// channel 解法：奇偶各一个 channel 传令牌
odd, even := make(chan struct{}), make(chan struct{})
go func(){ for i:=1;i<=100;i+=2 { <-odd; fmt.Println(i); even<-struct{}{} } }()
go func(){ for i:=2;i<=100;i+=2 { <-even; fmt.Println(i); odd<-struct{}{} } }()
odd <- struct{}{}      // 启动
// 实际要配合 WaitGroup/done 等结束
```

mutex+Cond 解法：共享计数 + `sync.Cond`，每个 goroutine 等到轮到自己（按奇偶判断）再打印并 `Broadcast`。

115. 带超时+限并发的批量执行器

```
func RunAll(ctx context.Context, tasks []func(context.Context) (int, error), limit int) ([]int, error)
{
    g, ctx := errgroup.WithContext(ctx)
    g.SetLimit(limit)                 // 限并发
    results := make([]int, len(tasks))
    for i, t := range tasks {
        i, t := i, t
        g.Go(func() error {
            v, err := t(ctx); if err != nil { return err } // 首个错误触发 ctx 取消
            results[i] = v; return nil
        })
    }
    if err := g.Wait(); err != nil { return nil, err }
    return results, nil
}
```

超时由外层 `ctx, cancel := context.WithTimeout(...)` 提供；errgroup 收集首个 error 并自动 cancel 其余。各 goroutine 写自己下标的 results，无需加锁。

116. singleflight

```
type call struct{ wg sync.WaitGroup; val interface{}; err error }
type Group struct{ mu sync.Mutex; m map[string]*call }
func (g *Group) Do(key string, fn func()(interface{},error)) (interface{}, error) {
    g.mu.Lock()
    if g.m == nil { g.m = map[string]*call{} }
    if c, ok := g.m[key]; ok { g.mu.Unlock(); c.wg.Wait(); return c.val, c.err } // 搭车等待
    c := &call{}; c.wg.Add(1); g.m[key] = c; g.mu.Unlock()
    c.val, c.err = fn()       // 只有第一个真正执行
    c.wg.Done()
    g.mu.Lock(); delete(g.m, key); g.mu.Unlock()
    return c.val, c.err
}
```

同一 key 并发请求只穿透一次，其余等结果——防缓存击穿。生产用 `golang.org/x/sync/singleflight`。

117. worker pool 优雅退出

```go
func Pool(n int, tasks <-chan func()) *sync.WaitGroup {
    var wg sync.WaitGroup
    for i:=0;i<n;i++ {
        wg.Add(1)
        go func(){ defer wg.Done(); for t := range tasks { t() } }() // tasks close 后 range 退出
    }
    return &wg
}
// 用法：close(tasks) 表示不再有任务；wg.Wait() 等已入队任务跑完
```

要点：用 `range channel` 接任务，生产者 `close(tasks)` 广播「无更多任务」，worker 把队列里剩余任务跑完后自然退出，`wg.Wait()` 确保全部完成。

N. 反射 reflect

118. 反射三大定律
① 接口值 → 反射对象：`reflect.TypeOf` / `reflect.ValueOf`。② 反射对象 → 接口值：`Value.Interface()`。③ 要修改反射对象，其值必须**可设置（settable / addressable）**。`reflect.Type` 描述类型元信息（名字、Kind、字段、方法），`reflect.Value` 持有具体值并能读/写（若可寻址）。

119. 反射改字段 / 为什么传指针

```go
v := reflect.ValueOf(&obj).Elem()       // 必须指针再 Elem，才可寻址
f := v.FieldByName("Name")
if f.CanSet() { f.SetString("x") }
```

传值给 `ValueOf` 拿到的是**副本**，不可寻址、`CanSet()` 为 false，改了也影响不到原对象——和「值传递」本质一致。必须传指针 `&obj`，`Elem()` 解引用得到指向原内存、可寻址的 Value，才能 Set。私有字段即使可寻址 `CanSet` 也为 false。

120. struct tag 解析 / 手写 mini Unmarshal
tag 是 struct 字段后的字符串字面量，`field.Tag.Get("json")` 解析。

```go
func Fill(m map[string]string, out interface{}) {
    v := reflect.ValueOf(out).Elem(); t := v.Type()
    for i := 0; i < t.NumField(); i++ {
        key := t.Field(i).Tag.Get("map")
        if val, ok := m[key]; ok && v.Field(i).CanSet() {
            v.Field(i).SetString(val)        // 简化：只处理 string 字段
        }
    }
}
// type S struct{ Name string `map:"name"` }; Fill(map[string]string{"name":"bob"}, &s)
```

121. 反射为什么慢
① 运行时动态解析类型、无法被编译器内联/优化；② 大量类型检查、装箱拆箱；③ `FieldByName` 走字符串匹配遍历；④ 常伴随堆分配。规避：缓存 `reflect.Type`/字段索引信息（只解析一次）、用代码生成（如 `easyjson`、protobuf 生成代码）在编译期替代运行时反射。

122. encoding/json 底层 / 私有字段
Marshal/Unmarshal 用反射遍历 struct 字段：读 tag 决定 JSON key、读字段类型决定编解码方式、用 `Value.Set*` 写回（Unmarshal）。**私有（小写）字段不被序列化**：反射无法读取/设置未导出字段的值（`CanSet` / `Interface` 受限，跨包访问私有字段违反封装），所以 json 包跳过它们。

O. 泛型深入

123. 类型约束 / type set
约束是一个 interface，规定类型参数必须满足的方法和/或类型集合。

- `any`：任意类型（即 `interface{}`）。
- `comparable`：支持 `==`/`!=` 的类型（可做 map key）。
- 自定义约束：`type Number interface{ ~int | ~float64 }`。
- **type set（类型集）**：1.18 起接口可由方法集**和**类型项（`int | string`）共同定义其「类型集合」，约束本质就是限定类型参数落在某个 type set 内。

124. `~` 的含义
`~int` 表示「底层类型是 int 的所有类型」，包括 `type MyInt int` 这种**自定义命名类型**。直接写 `int` 只匹配 int 本身，不含 MyInt。所以写库时常用 `~` 让约束覆盖用户的衍生类型。

125. 单态化 vs 字典传递
Go 泛型是**混合策略**：按 **GC shape**（内存布局/指针形状相同的类型）分组生成代码（GC shape stenciling），同一 shape 的多个类型**共享一份实例化代码**，并通过**字典（dictionary）**传入具体类型的元信息。所以不是 C++ 那种每个类型一份的纯单态化，也不是纯字典。影响：二进制体积比纯单态化小，但字典间接访问带来轻微运行时开销（指针类型尤其走字典，可能比手写具体类型慢）。

126. 泛型 vs 接口（举例）

- **泛型明显更优**：`func Max[T constraints.Ordered](a, b T) T` —— 类型安全、无装箱、返回具体类型；用 interface{} 得断言、丢类型、可能 panic。容器如 `Stack[T]` 同理。
- **不该用泛型**：需要异质集合（`[]io.Writer` 装不同实现）、或只关心行为不关心具体类型（`io.Reader` 作参数）——这是接口的主场，泛型反而把类型参数传染得到处都是。经验：「不同类型、相同算法」用泛型；「不同实现、相同抽象」用接口。

127. 为什么不支持泛型方法
Go 不允许在方法上额外引入类型参数（只能用接收者类型已声明的类型参数）。原因：泛型方法会让接口实现的判定变得极难——一个类型是否满足接口，需要对无穷多种类型参数实例化校验，无法静态确定，且与 Go 的接口隐式实现 + 运行时动态派发模型冲突（itab 无法为无限实例预生成）。限制了：不能在已有具体类型上加「任意类型参数的方法」，只能改用顶层泛型函数。

P. 闭包、方法值、标准库接口

128. 闭包 / 捕获值还是引用
闭包是函数 + 其引用的外层变量环境。Go 闭包捕获的是**变量本身（引用/地址）**，不是值快照——所以闭包内能修改外层变量、且看到后续变化。正因为闭包可能在外层函数返回后仍存活，被捕获的变量生命周期超出栈帧，**逃逸到堆**由 GC 管理。（这也是 1.22 前循环变量坑的根源。）

129. 方法值 vs 方法表达式

- **方法值** `t.M`：绑定了接收者 t 的函数值，类型是 `func(args)`，调用时不用再传接收者。会捕获 t（可能逃逸）。
- **方法表达式** `T.M`：未绑定接收者，类型是 `func(T, args)`，调用时**显式把接收者作第一个参数**传入。指针方法用 `(*T).M`。

130. io.Reader/io.Writer 为什么是典范
都是**单方法小接口**：`Read(p []byte)(n int, err error)` / `Write(p []byte)(n int, err error)`。好处：① 极易实现和满足，任意类型只要有这一个方法就能接入整个 io 生态；② **可组合**——`io.Copy`、`bufio`、`gzip`、`http` 全基于它们拼装，文件/网络/内存/加密流统一抽象；③ 符合「接口越小越通用」「面向行为而非实现」。是「小接口 + 组合」哲学的标杆。

131. io.Copy 工作原理 / 快路径

`io.Copy(dst, src)` 循环 `src.Read` 到 buffer 再 `dst.Write`，直到 EOF。优化快路径：① 若 src 实现了 `WriterTo`（`WriteTo(dst)`），直接调它；② 否则若 dst 实现了 `ReaderFrom`（`ReadFrom(src)`），调它。这些接口让具体类型可以走**零拷贝/系统调用优化**（如 `*os.File`/`net.TCPConn` 用 `sendfile`/`splice` 在内核态直接传数据，不经用户态 buffer）。

132. 手写 io.Reader 包装器（转大写）

```
type UpperReader struct{ r io.Reader }
func (u *UpperReader) Read(p []byte) (int, error) {
    n, err := u.r.Read(p)
    for i := 0; i < n; i++ {
        if p[i] >= 'a' && p[i] <= 'z' { p[i] -= 32 }
    }
    return n, err
}
// io.Copy(os.Stdout, &UpperReader{strings.NewReader("hello")}) -> HELLO
```

中间件式 wrapper：包住底层 Reader，在 Read 返回的数据上加工后透传——和 `gzip.NewReader` 等同一模式。（注：跨 ASCII/UTF-8 边界的字符处理需更严谨，这里仅演示 ASCII。）

Q. 底层、性能与版本特性

133. 内联 / 不能内联的函数 / 与逃逸的关系

内联：编译器把小函数体直接嵌入调用处，省掉函数调用开销并打开进一步优化。**不能内联**：函数体太大（超过内联预算）、含 `defer`/`recover`/`go`、闭包复杂、递归、含 `for range`（部分情况）等。与逃逸关系：内联后变量分析在调用者上下文进行，**可能让原本逃逸的变量不再逃逸**（栈分配），相互促进。查看：`go build -gcflags='-m'` 会打印 `can inline` / `inlining call to`。

134. map 扩容两种情况 / 渐进式 rehash

- **翻倍扩容**：装载因子过高（元素数/bucket 数 > 6.5），bucket 数翻倍，缓解碰撞。
- **等量扩容（same size）**：bucket 数不变，但 overflow bucket 过多（大量增删导致稀疏/碎片），重新整理把数据紧凑搬到新 bucket，回收 overflow。
- **渐进式 rehash**：扩容时不一次性搬完，保留 `oldbuckets`，每次对 map 的写操作（增/删/改）顺带 `evacuate` 搬迁一两个旧 bucket 到新 buckets，分摊开销避免单次卡顿。读时新旧都查。

135. sync.Map 底层 / 适用

内部两个 map：**read（只读，atomic 访问，无锁快路径）** 和 **dirty（含写，需加锁）**，外加 `misses` 计数。读优先走 read 无锁；read 未命中则加锁查 dirty 并 misses++；misses 累积到一定值，把 dirty 提升为 read。适合：**读多写少、key 集合相对稳定 / 增长后趋稳**（如缓存、只增配置）。不适合：**写频繁或 key 不断变化**——dirty 频繁重建、misses 频繁触发提升，反而比 `RWMutex+map` 慢，且内存占用更高。

136. unsafe.Pointer 四条规则 / uintptr

合法用法（Go 文档明确的几类）：① `*T1` ↔ `unsafe.Pointer` ↔ `*T2` 在内存布局兼容时互转；② `unsafe.Pointer` ↔ `uintptr` 做指针运算，但**必须在同一表达式内完成**、不可中途停留；③ 调用 `syscall.Syscall` 等需要 uintptr 参数时；④ 配合 `reflect.SliceHeader`/`StringHeader`（1.20 后推荐 `unsafe.Slice`/`String`）。**uintptr 不能长期持有**：它只是个整数、不被 GC 视为指针引用，GC 可能在它存活期间**回收或移动**（栈扩容会移动对象）所指对象，导致它指向无效内存。

137. unsafe 零拷贝 string ↔ []byte

```
func b2s(b []byte) string { return unsafe.String(unsafe.SliceData(b), len(b)) } // 1.20+
func s2b(s string) []byte { return unsafe.Slice(unsafe.StringData(s), len(s)) }
```

（1.20 前用 `*(*string)(unsafe.Pointer(&b))` 等老写法。）风险：① 转出的 string 与 []byte 共享内存，**改 []byte 会破坏 string 不可变性**，引发难查 bug；② 由 string 转出的 []byte **绝不能写**——string 可能位于只读内存段，写会 SIGSEGV；③ 要保证底层数组生命周期覆盖使用期，否则被 GC 回收成悬垂。仅在性能热点、且能严格保证只读时用。

138. 版本特性时间线

- **1.18 泛型**：解决重复写类型相似的容器/算法、或用 interface{} 丢类型的痛点。
- **1.20 `errors.Join`**：把多个 error 合并成一个（多错误聚合，配合 `Is`/`As` 遍历），解决「一次操作多处失败如何聚合上报」。
- **1.21 `min`/`max`/`clear` 内建 + `slices`/`maps` 包 + `slog`**：常用工具内建化、结构化日志进标准库。
- **1.22 loopvar 修复**：循环变量每轮新建，消除闭包/goroutine 捕获循环变量的经典 bug。

139. 1.22 循环变量修复 / 兼容性
1.22 前 `for` 的循环变量在整个循环**复用同一个**，闭包/goroutine 捕获到的是同一变量，跑时常拿到最后一个值（见 Q44）。1.22 改为**每轮迭代创建新的循环变量**，捕获到的是各轮独立的副本，坑自然消失。兼容性：理论上有极少数依赖「旧的共享变量语义」的代码行为会变，但这类代码几乎都是 bug；Go 用 `go.mod` 里的 **go 版本声明** 控制是否启用新语义（声明 `go 1.22+` 才启用），保证老模块行为不变——这是 Go「向后兼容」承诺下少见的语义变更，靠版本门控落地。

对照使用建议

1. **先盖答案自测**：每题先用 30 秒讲一遍主线，卡壳的标 ❌，再看答案。
2. **追问层是重点**：能背定义不算过，要能接住「为什么这么设计 / 换个设计会怎样 / 极端情况」。
3. **答错/答不全的题** → 按原题库附录计划，连同正确原理单独成篇 `go-internals-{子主题}-*.md` 深挖，并挂进 `CLAUDE.md` 第六节复习调度（+1 / +7 / +30 天）。
4. **手感区（A/E/F）也别轻敌**：天天写不代表讲得清，重点验证 13、24、33、44、54 这类「经典坑」。
5. **盲区（I/J/Q）优先动手**：调度器和 GC 平时碰不到底层，建议配合 `GODEBUG=schedtrace=1000` / `gctrace=1`、pprof 实际观测一遍，把文字变成手感。

附录：自我思考

- 这份答案是「对照基线」而非终点。真正的毕业标准是**盖住它能讲出来**，而不是「看懂了」。看懂和讲出来之间，正是这份题库要暴露的那层 gap。
- 答案里凡涉及具体数字（栈 2KB、bucket 8 槽、装载因子 6.5、cache line 64B、扩容阈值 256/1024、STW <1ms）——**记机制和量级，不必死抠精确值**，面试讲清「为什么是这个量级」比背数字更重要，且这些数字会随版本微调。
- 几个最该亲手验证的点：① `-gcflags=-m` 看逃逸；② `-race` 跑一个并发计数器看报错；③ `GODEBUG=gctrace=1` 看 GC 节奏与 STW；④ 写个纯死循环 goroutine 在不同 GOMAXPROCS 下观察（验证抢占）；⑤ benchmark 对比 `+` vs `strings.Builder`。把这 5 个跑通，I/J/K 三类的「文字理解」就落地成「手感」了。
- 下一步沉淀顺序建议：**GMP（I）→ GC（J）→ 逃逸/性能（K）** 这三块盲区性价比最高，且互相咬合；手感区 A/E/F 只需针对「经典坑」补漏，不必从头写笔记。